

Group name: 404 NOT FOUND

Members: John Elmer Bobier Jr., Charls Emil C. Barquin, Christian Jeric Non, Rod Rañola, Vincent Macauyam

Course Yr. & Block: BSIS - 2A

Problem Number: 1

Problem Statement

Remove the Add Quantity Input if the Default Cart Item Qty is 1.

CODE CHANGES

Old Cart.php vs the New cart.php

We added select all/select checkbox:

```
<td><input type="checkbox"
name="selected_items[]" value="<?php echo
$row['id']; ?>" class="item-checkbox"
data-price="<?php echo $row['price'] *
$row['quantity']; ?>"></td>
```

And the major issue fix:

```
<form action="checkout-system.php"
method="post">
```

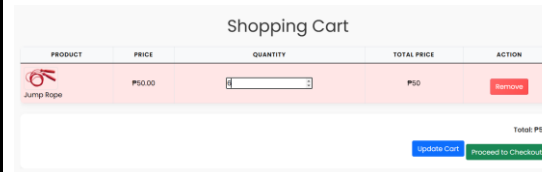
This is the reason why items were not passed from cart to checkout

Above are the changed that make the cart functionality works

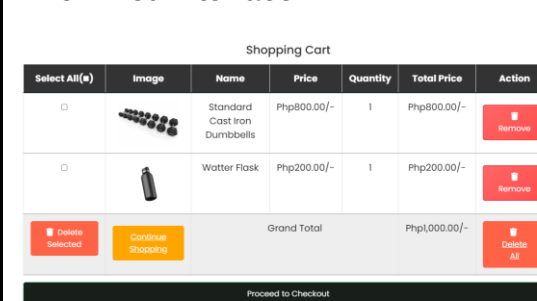
This is the link see what is the full difference in cart.php:
<https://www.diffchecker.com/nud52pv6/>

INTERFACE CHANGES

Old cart interface.



Error fixed interface.



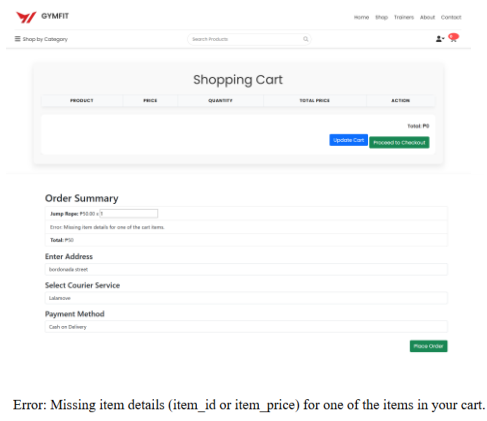
Describe the functionality added or Changed

The updated shopping cart has a better interface and now lets you handle multiple items at once, which makes it more convenient. You can't edit the quantity directly in the cart anymore, but we added some useful features like a "Select All" option to choose all items at once, buttons to "Delete Selected" or "Delete All" items quickly, and a "Continue Shopping" button to go back and add more items. These changes make the cart easier and more practical to use.

- Redesigned interface for a more organized look.
- Added support for handling multiple items in the cart.
- Removed the ability to edit quantities directly in the cart.
- Introduced a "Select All" option for

		bulk item selection. - Added "Delete Selected" and "Delete All" buttons for easier item removal. - Included a "Continue Shopping" button for seamless navigation.
--	--	--

Problem Number: 2
Problem Statement
Unable to Checkout. Need to make the Checkout Work.

CODE CHANGES	INTERFACE CHANGES	Describe the functionality added or Changed
First we started with changes in cart.php we started using <pre>\$user_id = \$_SESSION['user_id'];</pre> And <code><form action="checkout-system.php" method="post" id="cart-form"></code> in order to passed the items in cart to checkout-system.php <i>This is the link see what is the full difference in cart.php:</i> https://www.diffchecker.com/nud52pv6/	Old non-functional checkout interface 	The cart view is way simpler now, focusing on the essentials like product names, prices, quantities, and totals, which makes it easy to see what you're buying without any extra clutter. There's also a new error message that pops up if any item details are missing, helping users catch mistakes before they finish their orders. The action buttons, like "Place Order" and "Proceed to Checkout," are much clearer and stand out more, so it's easy to figure out what to do

Next we created a new checkout-system.php

This is the link see what is the checkout-system.php:
https://drive.google.com/file/d/1E5NnH2ZiGvAp2_cj-BW0DSvFJ1Y8CGTx/view?usp=sharing

Also the last added is
check_orderdetails.php
Which gives you a receipt && send you
an Email of that receipt.

This is the link see what is the check_orderdetails.php:
https://drive.google.com/file/d/1E5NnH2ZiGvAp2_cj-BW0DSvFJ1Y8CGTx/view?usp=drive_link

Error fixed interface.

The screenshot displays a web application interface for a shopping cart and checkout process. At the top, there is a navigation bar with the 'EMART' logo and links for Home, Shop, Wishlist, About, and Contact. Below the navigation bar is a search bar and a user profile icon. The main content area is divided into two sections. The upper section, titled 'Shopping Cart', shows a table with columns for Item ID, Image, Name, Price, Quantity, Total Price, and Action. It lists two items: 'Kardus Box' and 'Kardus Karton', both priced at PHP100.00. Below the table are buttons for 'Add to Cart', 'Update Cart', and 'Checkout'. The lower section, titled 'Complete Your Order', contains a form for user details and payment information. It includes fields for Name, Contact Number, Payment Method (Cash on Delivery), Island, City, Province, and Street. A large red 'Order' button is at the bottom of the form. Below the form is a 'THANK YOU FOR SHOPPING!' message, a receipt summary, and a 'CONTINUE SHOPPING' button.

Item ID	Image	Name	Price	Quantity	Total Price	Action
1		Kardus Box	PHP100.00	1	PHP100.00	Remove
2		Kardus Karton	PHP100.00	1	PHP100.00	Remove
Grand Total					PHP200.00	Checkout

Complete Your Order

Kardus box (1)
Grand Total: Php (1018)

Your Name: Your Number:

Payment Method: Courier Service:

Island: Province:

City: Street:

[Order](#)

THANK YOU FOR SHOPPING!

Kardus box (1)
Total: Php1095,-

Your Name: John Elmer S. Balar Jr
Your Number: 12345678
Courier: J&T Express
Your Address: Bontolada, zed, multiplot, multiplot, Luzon
Your Payment Mode: cash on delivery
Tracking Number: EBRUVEUHO

Receipt has been sent to your email 📧

[CONTINUE SHOPPING](#)

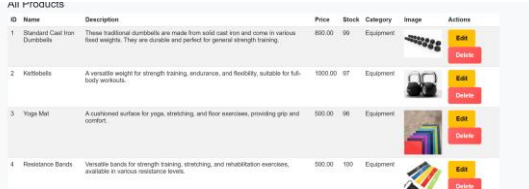
next. Plus, the overall design has taken on a minimalist vibe, with fewer visual distractions, which really improves usability.

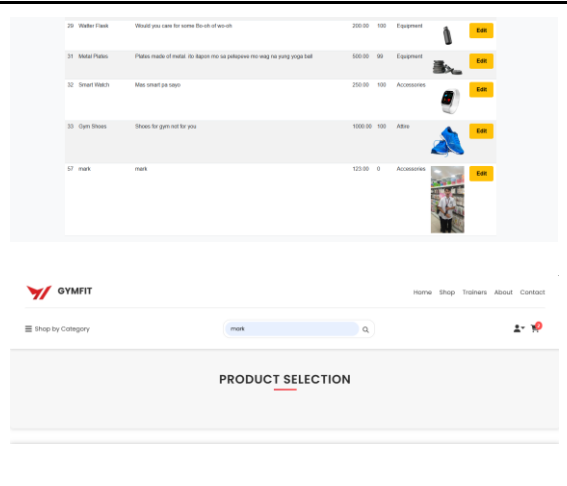
- Simplified Cart View
- Error Message
- Prominent Action Buttons
- Minimalist Design

Problem Number: 3

Problem Statement

Troubleshoot the delete button in Admin, though you need to make the delete, "Not to delete the entire item record as it will have an issue retrieving the past orders for that item.

CODE CHANGES	INTERFACE CHANGES	Describe the functionality added or Changed
We remove the entire delete functionality. <pre>// Delete Product Logic if (isset(\$_GET['delete_id'])) { \$delete_id = \$_GET['delete_id']; \$sql_delete = "DELETE FROM items WHERE item_id = \$delete_id"; if (\$conn->query(\$sql_delete) === TRUE) { \$manage_product_message = "Product deleted successfully."; } else { \$manage_product_message = "Error deleting product: " . \$conn->error; } }</pre> <i>This is the link see what is the full difference in cart.php:</i> https://www.diffchecker.com/DlvoPVud/	Old non-functional checkout interface  Error fixed interface.	The previous version of the system included a button designed to delete products, but it caused errors during the process. To resolve this issue, we adopted a different approach based on product stock levels. When the stock of a product reaches zero, the product no longer appears on the index or client-facing page. Importantly, the system does not delete the product from the records; rather, it simply hides it from view when unavailable. This approach ensures that client purchase records are preserved while also preventing out-of-stock products from being displayed.

		
--	--	--

Problem Number: 4
Problem Statement
System must land in the Index as a Landing page.

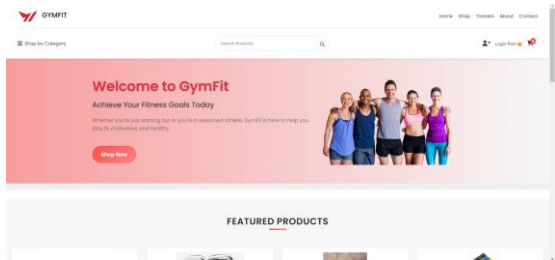
CODE CHANGES	INTERFACE CHANGES	Describe the functionality added or Changed
http://localhost/Gym-fit%20website/GYMFIT-INTERFACE2/php/	Old non-functional checkout interface	We tried adding “.htaccess” that contains “DirectoryIndex index.php” but the basic fix to this problem is to search the folder name along with the php folder to display the index or landing page of our website.

Index of /Gym-fit

Name	Last modified	Size	Description
 Parent Directory		-	
 admin/	2024-12-15 14:28	-	
 css/	2024-12-15 14:28	-	
 js/	2024-12-15 14:28	-	
 php/	2024-12-10 17:28	-	
 product_img/	2024-12-15 14:28	-	
 res_img/	2024-12-15 14:28	-	
 sign-in_sign-up/	2024-12-15 14:28	-	
 trainer_img/	2024-12-15 14:28	-	

Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12 Server at localhost Port 80

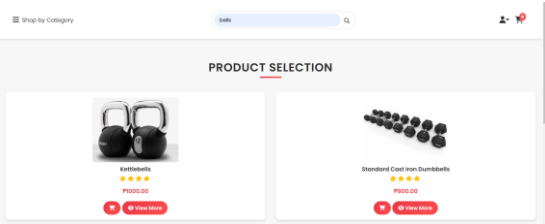
Error fixed interface.



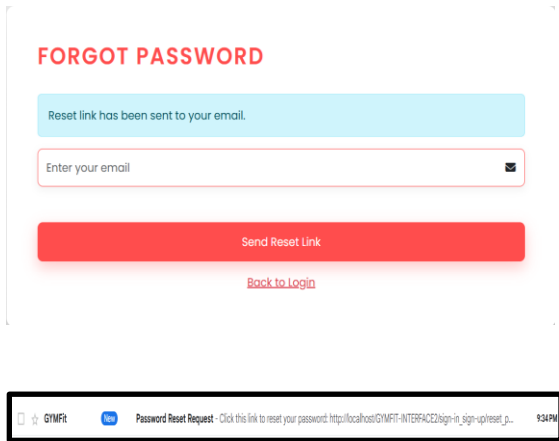
Problem Number: 5

Problem Statement

Troubleshoot the delete button in Admin, though you need to make the delete, "Not to delete the entire item record as it will have an issue retrieving the past orders for that item.

CODE CHANGES	INTERFACE CHANGES	Describe the functionality added or Changed
<pre> if (isset(\$_GET['search']) && !empty(\$_GET['search'])) { \$search_term = mysqli_real_escape_string(\$conn, \$_GET['search']); // SQL query to search for products by name or description \$sql = "SELECT * FROM items WHERE item_name LIKE '%\$search_term%' OR item_desc LIKE '%\$search_term%' ORDER BY item_name ASC"; } else if (isset(\$_GET['category_id'])) { // Display products by category \$category_id = \$_GET['category_id']; \$sql = "SELECT * FROM items WHERE category_id = \$category_id"; } else { // Default to showing all products \$sql = "SELECT * FROM items ORDER BY rand() "; } \$all_products = \$conn- >query(\$sql); ?> </pre>	Error fixed interface. 	The added functionality introduces a dynamic filtering mechanism for product display based on user input or category selection. It prioritizes user-driven search by checking if a search term is provided (<code>\$_GET['search']</code>), sanitizing the input to prevent SQL injection, and querying products whose names or descriptions match the search term. If no search term is entered but a category ID is provided (<code>\$_GET['category_id']</code>), it retrieves products specific to that category. In the absence of both filters, the code defaults to displaying all products in a randomized order. This enhancement ensures a tailored user experience, allowing flexible browsing and refined product discovery.

Problem Number: 6
Problem Statement
Troubleshoot the delete button in Admin, though you need to make the delete, "Not to delete the entire item record as it will have an issue retrieving the past orders for that item.

CODE CHANGES	INTERFACE CHANGES	Describe the functionality added or Changed
<pre> if (\$_SERVER['REQUEST_METHOD'] == 'POST') { \$email = \$_POST['email']; // Check if email exists \$sql = "SELECT * FROM user_account WHERE email='\$email'"; \$result = mysqli_query(\$conn, \$sql); if (mysqli_num_rows(\$result) == 1) { // Generate a unique reset token \$token = bin2hex(random_bytes(50)); // Save token in the database \$sql = "UPDATE user_account SET reset_token='\$token' WHERE email='\$email'"; mysqli_query(\$conn, \$sql); // Create the reset link </pre>	Error fixed interface. 	The provided code handles the initial steps of the password reset process for a user requesting to reset their password. When the user submits their email through a POST request, the system checks if the email exists in the database by querying the <code>user_account</code> table. If the email is found, a unique reset token is generated using <code>random_bytes()</code> and stored in the database. A reset link is then created by appending the token as a query parameter to the URL. Finally, the code initializes PHPMailer, which will be used to send the reset link to the user's email address (though the email sending process is not fully implemented in the provided snippet). This functionality ensures that users can securely reset their passwords through a token-based verification system.

<pre><i>\$reset_link =</i> "localhost/GYMFIT- INTERFACE2/sign-in_sign- up/reset_password.php?token=" . \$token; // Initialize PHPMailer \$mail = new PHPMailer(true);</pre>		
--	--	--

https://drive.google.com/file/d/1_t_8lrdVTnNgd7VL6EhmuKdvsnTgpflU/view?usp=sharing

Group Name: 404 NOT FOUND

Name: John Elmer B. Bobier Jr

Role: Front-end Developer

Frontend Code Contributions:

- *Cart Interface*
- *Checkout Interface*
- *Designer Interface*
- *Admin Orders Interface*
- *Admin Landing Page Interface*
- *Tester*